UNIT –II LOOPING, CONTROL STRUCTURES, FUNCTIONS, RECUSRION IN R

1) R Programming Structures

1. Write an R program to take a numeric vector and print its length, type (typeof), and class.

```
# 1
v <- c(10,20,30,40,50)
print(length(v))
print(typeof(v))
print(class(v))
```

2. Write an R program to create a list containing a vector, matrix, and data frame, then display the structure using str().

```
# 2
v <- c(1,2,3)
m <- matrix(1:6, nrow=2)
df <- data.frame(Name=c("A","B"), Age=c(20,21))
lst <- list(v,m,df)
str(lst)
```

2) for() Loop

3. Write an R program using for loop to print numbers from 1 to 10 and their squares.

```
# 3
for(i in 1:10){
```

```
  cat(i, i^2, "\n")
}
```

4. Write an R program using for loop to find the sum of even numbers from 1 to 50.

```
# 4
sum <- 0
for(i in 1:50){
  if(i %% 2 == 0)
    sum <- sum + i
}
print(sum)
```

3) while() Loop

5. Write an R program using while loop to print numbers from 10 down to 1.

```
# 5
i <- 10
while(i >= 1){
  print(i)
  i <- i - 1
}
```

6. Write an R program using while loop to find the factorial of a number n.

```
# 6
n <- 5
fact <- 1
while(n > 0){
  fact <- fact * n
  n <- n - 1
}
print(fact)
```

4) Control Statements (break, next)

7. Write an R program that prints numbers 1 to 20 but skips multiples of 3 using next.

```
# 7
for(i in 1:20){
  if(i %% 3 == 0)
    next
  print(i)
}
```

8. Write an R program that scans a vector and stops when it finds a negative number using break.

```
# 8
v <- c(4,8,9,-3,5,7)
for(i in v){
```

```
  if(i < 0)
    break
  print(i)
}
```

5) Looping Over Non-Vector Sets (lists)

9. Create a list of 5 items and use a loop to print each element.

```
# 9
lst <- list(10,"R",TRUE,3.5,c(1,2,3))
for(i in lst){
  print(i)
}
```

10. Create a list containing vectors of different lengths and use a loop to print the length of each item.

```
# 10
lst <- list(c(1,2), c(3,4,5), c(6), c(7,8,9,10))
for(i in lst){
  print(length(i))
}
```

6) If-Else

11. Write an R program using if-else to check whether a number is positive, negative, or zero.

```
# 11
n <- -5

if(n > 0)
  print("Positive")
else if(n < 0)
  print("Negative")
else
  print("Zero")
```

12. Write an R program using if-else to assign grades: A (≥90), B (≥75), C (≥60), else Fail.

```
# 12
mark <- 82

if(mark >= 90)
  print("A")
else if(mark >= 75)
  print("B")
else if(mark >= 60)
  print("C")
else
```

```
  print("Fail")
```

7) Arithmetic Operators

13. Write an R program that reads two numbers and prints + , − , × , ÷ , %% , %/% results.

```
# 13

a <- 20

b <- 6


print(a + b)

print(a - b)

print(a * b)

print(a / b)

print(a %% b)

print(a %/% b)
```

14. Write an R program to calculate simple interest using arithmetic operators.

```
# 14
p <- 10000
r <- 5
t <- 2

si <- (p * r * t) / 100
print(si)
```

8) Boolean Operators & Values

15. Write an R program that checks whether a number lies between 10 and 50 using

& and |.

```
# 15
n <- 35

print(n >= 10 & n <= 50)
print(n < 10 | n > 50)
```

16. Write an R program to filter values in a vector that satisfy: x > 5 AND x is even.

```
# 16
x <- c(2,4,6,7,8,10,11)

print(x[x > 5 & x %% 2 == 0])
```

9) Default Values for Arguments

17. Write a function power(x, p=2) that returns x^p; call it with and without p.

```
# 17
power <- function(x, p = 2){
  x^p
}

print(power(5))
print(power(5,3))
```

18. Write a function discount(price, rate=0.10) that returns final price after discount.

```
# 18
discount <- function(price, rate = 0.10){
  price - (price * rate)
}

print(discount(1000))
print(discount(1000,0.20))
```

10) Return Values (implicit vs explicit return)

19. Write a function that returns the largest of two numbers (use explicit return()).

```
# 19
largest <- function(a,b){
  if(a > b)
    return(a)
  else
    return(b)
}

print(largest(10,20))
```

20. Write a function that returns the sum of a vector without using return() (implicit

return).

```
# 20
sumVector <- function(v){
  sum(v)
}

print(sumVector(c(1,2,3,4,5)))
```

11) Deciding Whether to Explicitly call return()

21. Write a function that checks if a number is prime and uses return(FALSE) early when divisible.

```
# 21
isPrime <- function(n){
  if(n <= 1)
    return(FALSE)

  for(i in 2:(n-1)){
    if(n %% i == 0)
      return(FALSE)
  }

  return(TRUE)
}
```

```
print(isPrime(17))
```

22. Write a function that calculates average and returns "Invalid" using return() if the vector is empty.

```
# 22
average <- function(v){
  if(length(v) == 0)
    return("Invalid")

  mean(v)
}

print(average(c(10,20,30)))
```

12) Returning Complex Objects

23. Write a function that returns mean and standard deviation as a list.

```
# 23
stats <- function(v){
  list(
    mean = mean(v),
    sd = sd(v)
  )
}
```

```
print(stats(c(10,20,30,40,50)))
```

24. Write a function that returns min, max, range as a named list.

```
# 24
details <- function(v){
  list(
    min = min(v),
    max = max(v),
    range = range(v)
  )
}

print(details(c(10,20,30,40,50)))
```

13) Functions are Objects (First-class)

25. Assign a function to a variable and call it (e.g., f <- function(x) x*2).

```
# 25
f <- function(x){
  x * 2
}

print(f(10))
```

26. Write a function that accepts another function as an argument and applies it to a vector.

```
# 26
applyFunction <- function(fun, v){
  fun(v)
}

print(applyFunction(sum, c(1,2,3,4,5)))
```

14) No Pointers in R

27. Write a program showing pass-by-value behavior: modify a vector inside a function and show original remains unchanged.

```
# 27
modifyVector <- function(x){
  x[1] <- 100
  print(x)
}

v <- c(1,2,3,4)
modifyVector(v)
print(v)
```

28. Write a program using <<- to modify a global variable inside a function and display the change.

```
# 28
x <- 10

change <- function(){
  x <<- 50
}

change()
print(x)
```

15) Recursion

29. Write a recursive function to compute factorial(n).

```
# 29
factorialRec <- function(n){
  if(n == 0)
    return(1)

  n * factorialRec(n-1)
}

print(factorialRec(5))
```

30. Write a recursive function to compute Fibonacci(n).

```
# 30
fibonacci <- function(n){
  if(n <= 1)
    return(n)
```

```
  fibonacci(n-1) + fibonacci(n-2)
}

print(fibonacci(6))
```